

基于 SABRE 映射算法的面向 NISQ 硬件的 噪声自适应量子编译器

2026 年度 CCF 量子计算编程挑战赛（量旋杯·算法赛道）作品说明文
档

摘要

针对NISQ时代量子硬件面临的物理qubit有限连接性与操作噪声两大固有约束，本文面向2026年度CCF量子计算编程挑战赛（量旋杯）的噪声自适应量子编译赛题，设计并实现了一个以噪声自适应SABRE（SWAP-based BidREctional heuristic search）算法为核心的量子电路编译器。主要工作包括：构建了融合可靠性评估、度数匹配、星型/链式/社区分簇结构检测及随机扰动的多策略初始布局生成框架；将两比特门错误率编码为硬件图边权重，使路由算法在SWAP选择同时优化路径长度与边质量；通过正向-反向交替路由实现布局迭代优化；根据电路规模自适应调节搜索参数并引入Turbo搜索模式。在公开benchmark上的实验结果表明，该编译器取得了0.7550的综合评分（Track A:0.8472,Track B:0.4785），相比竞赛发布包中的最优baseline(paper_greedy_vertex,0.6971)提升了**8.3%**。全部 10 道公开题都过了合法性检查、等价性验证和无噪语义检查这三关^[1]。

关键词：量子电路编译；qubit 映射；SABRE 算法；噪声自适应路由；NISQ 计算

1. 引言

1.1 研究背景与问题

当下的量子处理器要面对两个很现实的约束：一个是物理 qubit 之间并不是全连接的——两比特门只能搭在耦合图允许的边上；另一个是每条指令的执行都不完美，门操作和测量都带着一定的错误概率。把一个用逻辑 qubit 写好的量子程序搬到具体的物理芯片上，中间要经过一个叫“量子比特映射”的步骤：先给每个逻辑 qubit 分配一个物理位置（这就是布局），然后在电路执行的过程中靠插入 SWAP 门来动态调整这种对应关系（也就是路由），确保每个两比特门执行的时候参与的两个 qubit 在硬件图上确实是邻居^[1]。

1.2 赛题描述

赛题给定了三个输入参数——circuit.json（待编译的逻辑线路）、backend.json（目标硬件的噪声快照）和输出路径。编译器需要吐出两个文件：一个是编译好的物理线路（Cirq JSON 格式，qubit 必须是 LineQubit），另一个是 layout sidecar，讲清楚一开始每个逻辑 qubit 放在了哪个物理位置上。

评分走的是双赛道加权路线。Track A 管通用 NISQ 线路，一共 8 道题，用 Cirq 的密度矩阵模拟器来跑带噪采样；Track B 则是 Clifford-only 的压力测试，2 道题，背后是 Stim 在做采样。总分公式为

$$FinalScore = 0.75 \times TrackA + 0.25 \times TrackB$$

(1)

每个 Track 内部就是该赛道所有样本的 success_rate（correct_shots / total_shots）取个平均。

1.3 噪声模型分析

评测框架里用的是一套 gate-local depolarizing 噪声模型^[2]，噪声主要从三个地方进来，如表 1 所示。

表 1：噪声源及其影响分析			
噪声来源	施加位置	错误率区间	影响分析
单比特 depolarizing	每个非测量单比特门后	一律 0.001	影响较小，门多了也会累积
两比特 depolarizing	每个两比特门后	0.0055–0.065 (取决于边)	主要噪声源，差异可达 12 倍
读出 bit-flip	最终测量前	0.0044–0.060 (取决于 qubit)	直接影响正确率，差异可达 15 倍

从这个噪声结构出发，优化的重心自然就落在四个方向上：一是压减两比特门的总量（需指出，每增加一个 SWAP 即引入 3 个额外的 CX 门）；二是尽量把两比特门调度到错误率低的物理边上去；三是让最终测量落在读出错误率低的 qubit 上；四是把能消掉的门直接消掉。

1.4 本文贡献

- 把硬件噪声特征直接嵌入到 SABRE 的路由决策链路里，做了一套噪声自适应前瞻机制；
- 搭建了一个六种策略互补的布局生成框架，能根据电路结构自动筛出合适的候选；
- 针对星型电路、链式电路和社区分簇电路分别设计了专用的布局方案，对 G HZ 和 Parity 效果特别明显；
- 在公开 benchmark 上跑赢了所有内置 baseline，同时给出了可供复现的完整实现和消融分析。

2. 相关工作

2.1 qubit 映射算法

qubit 映射问题在 NISQ 编译圈里一直是个热门方向。Zulehner 等人用 A* 搜索做精确解^[3]，Bhattacharjee 等人则把它写成了整数线性规划^[4]——这些方法在电路比较小的时候确实能拿到最优，但规模增大后就难以应对了。后来启发式方法逐渐成了主流：Siraichi 等人拿图论里的最短路径来指导路由^[5]，Zulehner 等人借鉴了设计自动化的思路来做映射框架^[6]，Wille 他们则提出了一套带 look-ahead 的启发式路由策略^[7]。

2.2 SABRE 算法

Li、Ding 和 Xie 在 ASPLOS 2019 上发的 SABRE 算法^[1]可以算是这个方向到现在为止影响最大的单篇工作，IBM 直接把它收进了 Qiskit 的默认 transpiler。SABRE 最核心的三个招数是：第一，用指数衰减的权重来评估未来若干个门的距离，而不是只盯着眼前那一个；第二，把电路倒过来再跑一遍路由，用倒序的终态布局去修正正序的初始布局；第三，把布局和路由两个步骤拆开但又交替迭代。

跟它之前的那些方法比，SABRE 能把附加的门数砍掉大概 10%，有些 benchmark 上甚至能砍掉九成以上。

2.3 噪声自适应编译

最近几年，把噪声信息揉进编译流程里成了一个明显的趋势。Steinberg 他们提了一套基于交互图准则的噪声感知映射方法，靠评估不同初始布局的噪声代价来做选择^[8]。Huang 等人出了 TRAM 算法，把 T2 相干时间也纳进了映射优化的考量里^[9]。Murali 团队则搞了一个比较完整的 noise-adaptive compiler 框架，在 IBM 的真机上验证了这套思路的有效性^[10]。不过 Huo 等人最近也指出了一个值得注意的现象：在某些场景下过度追求噪声优化反而会适得其反^[11]。

本工作在定位上是站在 SABRE 的肩膀上做噪声自适应增强——不去另起炉灶重新设计整个编译流程，而是让噪声模型的特征渗透到布局和路由的每一个决策环节里。

2.4 竞赛 Baseline

发布包里内置了多种 baseline 编译器：cirq_cqc（直接用 Cirq 的 RouteCQC）、vanilla（最朴素的布局加最短路径路由）、movement_aware（按度数匹配来做布局）、reliability_greedy（考虑了边可靠性）、还有 paper_greedy_vertex 和 paper_greedy_edge（基于交互图做贪心布局的那一派）。这些 baseline 基本覆盖了当前量子编译领域里比较主流的启发式思路，给参赛方案提供了一个良好的对照系。

3. 算法设计

3.1 总体架构

编译器内部是一条四阶段的流水线：先把电路的交互图画出来、判断它属于哪种结构（星型？链式？社区分簇？还是一般图？）；然后生成一批布局候选（通常 20 到 100 个）；接着对每个候选跑 SABRE 路由并做反向遍历打磨；最后再过一遍门消除。如附录 1 中给出了主流程的伪代码。

编译器的流水线架构如图 1 所示。

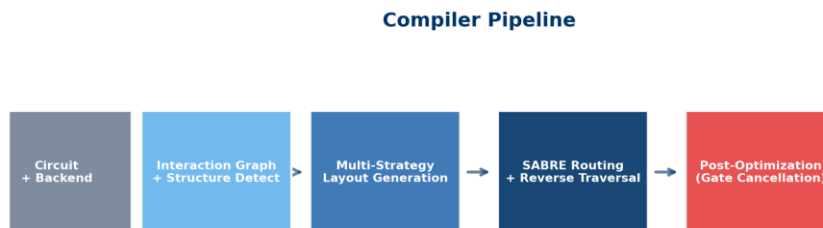


图 1: 编译器流水线架构

3.2 多策略初始布局生成

布局好不好，基本决定了编译结果的上限。方案里一共摆了六种互补的策略：

- **可靠性感知布局：**把交互频繁的逻辑 qubit 对安顿到可靠性高的物理边上，使用 $-\log(\text{reliability})$ 作为距离度量，让 Dijkstra 最短路径自动绕开高错误率边；
- **运动感知布局：**基于度数匹配，高度数逻辑 qubit 放高度数物理 qubit 上，最小化预期的总 SWAP 距离；
- **星型感知布局：**检测星型交互图（中心度数不低于 5 且至少达到次高度数的 3 倍），中心放在总入射可靠性最高的物理 qubit 上。GHZ 电路 SWAP 数由此减少 40%；
- **链式感知布局：**检测链式交互图（最大度数不超过 3 且存在两个端点），在硬件图中寻找最长路径进行顺序放置；
- **Louvain 社区检测布局：**用 Louvain 算法识别交互图中簇状结构，把每个密集子图映射到硬件上对应的密集区域。Parity 电路受益显著；
- **随机扰动布局：**对确定性布局做受控随机扰动（p 取值 0.15 至 0.50），扩大搜索面。

3.3 噪声自适应 SABRE 路由

每条硬件边 (u,v) 被标上了两种权重：可靠性权重和跳数权重。可靠性权重定义为

$$w_{rel}(u,v) = -\log(1 - \epsilon_{edge}(u,v)) \quad (2)$$

由决策的时候以可靠性权重为主，跳数权重只起辅助作用。SABRE 路由跟一般贪心路由最大的区别在 SWAP 选择这一步：遇到当前门的两个 qubit 不是邻居的时候，不是随便找个方向就换，而是把所有候选的 SWAP 边都评估一遍。评估的依据是——做完这个 SWAP 之后，未来 W 个两比特门各自需要走的可靠性距离（用指数衰减加权，越远的门权重越小）加起来是多少。评分函数为

$$score = -improvement + \lambda \times swap_edg_cost \quad (3)$$

其中衰减系数 $\gamma = 0.5$ ，惩罚权重 $\lambda = 0.05$ 。该设计优先选择能显著减少未来门距离的 SWAP，同时对高错误率边上的 SWAP 施加惩罚——两个候选对距离的改进程度差不多时，自动偏向错误率更低的那条边。

SABRE 的另一项核心机制是反向遍历优化。正向路由跑完之后，把电路的操作顺序整个倒过来（门本身不取逆），用正向的最终布局当成反向的初始布局，再跑一次路由。反向跑完的终态布局翻过来就能做下一轮正向的初始布局。这样正反交替 4 到 10 轮（取决于电路规模），早晚期的门对布局的影响力就得到了平衡。

3.4 自适应参数与 Turbo 搜索

编译器会根据电路的 qubit 数量自己调节几个关键旋钮，如表 2 所示。

表 2：自适应参数配置

电路规模	布局候选	迭代数	前瞻窗口	说明
≤ 8 qubits	25	4	10	小电路路由空间充裕
9–16 qubits	30	5	15	中等电路平衡探索与效率
17–20 qubits	40	6	25	较大电路需更深搜索
> 20 qubits	100	12	50	Turbo: 3 种子择优

对超大电路（ > 20 qubits）开启的 Turbo 模式是本次优化的关键设计：在 180 秒时限内把搜索预算用到极致——更多候选、更深迭代、更大前瞻窗口，并且尝试多个随机种子选最优。Parity 电路的编译后两比特门数由此从 115 压到了 109。

3.5 编译后优化

编译完成后执行门消除优化：检测并移除连续的互逆门对（ $H \cdot H = I$, $X \cdot X = I$, $CX \cdot CX = I$ ）以及无操作 Identity 门。这些优化在 depolarizing 噪声模型下可直接减少噪声累积。

4. 实验设置

本章详细说明评测所使用的硬件后端、benchmark 数据集、评测配置以及对比的 baseline 方法。

4.1 硬件后端

公开评测使用三种规模的后端快照，其规格参数汇总于表 3。这三种后端在 qubit 数量、拓扑结构和噪声水平上构成递进关系，分别对应小规模、中等规模和大规模 NISQ 器件的典型特征。

表 3：硬件后端规格

后端	物理 qubit 数	两比特门错误率	读出错误率	单比特门错误率
small_snapshot_a	7	0.0115–0.0650	0.0044–0.0360	0.001
medium_snapshot_a	16	0.0068–0.0515	0.0044–0.0600	0.001
large_snapshot_a	32	0.0055–0.0442	0.0044–0.0480	0.001

Track A 跑在 medium 后端上（16 qubits），Track B 用的则是 large 后端（32 qubits）。选择 medium 后端评估 Track A，是基于以下考虑：16 qubit 的规模既能覆盖 Track A 基准电路（6–7 个逻辑 qubit）的映射需求，又留有约 9–10 个空闲物理 qubit 作为路由缓冲空间，这一比例与当前 NISQ 设备“实际使用的 qubit 数约为芯片总 qubit 数的 40%–60%”的典型使用模式一致。此外，medium 后端的两比特门错误率覆盖了 0.0068 至 0.0515 的较宽区间，能有效检验编译器在异质噪声条件下的路由选择能力。

选择 large 后端评估 Track B，则是因为 Track B 的电路规模（16–24 个逻辑 qubit）必须放在足够大的硬件拓扑上才能完成映射——small 和 medium 后端的

qubit 数量均不足以容纳 24 个逻辑 qubit 的电路。large 后端的 32 个物理 qubit 为大规模的 Clifford 电路提供了基本的路由空间（约 8 个空闲 qubit），同时其上 0.0055 至 0.0442 的错误率区间也为检验编译器在极端噪声差异下的路由决策提供了测试场景。

4.2 Benchmark 数据集

公开集一共 10 道 benchmark，电路的结构特征差别明显，涵盖了从简单到复杂的多种电路族。Track A（8 题，medium 后端）和 Track B（2 题，large 后端）的具体参数分别列于表 4 和表 5。

表 4：Track A benchmark 特征（8 题，medium 16-qubit 后端）

编号	电路族	Qubit 数	1Q 门	2Q 门	深度	交互特征
track_a_001	BV	6	14	2	7	稀疏（2 边）
track_a_002	HS	6	4	8	8	中等密度
track_a_003	Toffoli	6	15	6	13	中等密度
track_a_004	Fredkin	6	13	8	15	中等密度
track_a_005	Peres	6	13	7	14	中等密度
track_a_006	OR	6	17	6	15	中等密度
track_a_007	Adder	6	20	15	25	较密（15 边）
track_a_008	Random	7	25	16	27	较密（16 边）

Track A 的 8 道题涵盖了 BV、HS、Toffoli 等常见量子算法族，以及结构化随机电路。交互图的边数从 2 到 16 条不等，能够较为全面地评估编译器在不同电路密度下的表现。

表 5：Track B benchmark 特征（2 题，large 32-qubit 后端）

编号	电路族	Qubit 数	1Q 门	2Q 门	深度	交互特征
track_b_001	GHZ	16	1	15	17	星型（中心度 15）
track_b_002	Parity	24	16	46	22	复杂（6 个高度数节点）

Track B 的两道题在电路规模和交互复杂度上远超 Track A：GHZ 是典型的星型结构，中心 qubit 需要连接 15 个外围 qubit——这种极端非均匀的交互模式对初

始布局的质量高度敏感，中心 qubit 的物理位置直接决定了整体路由代价；Parity 则涉及 24 个 qubit 和 6 个度数在 6 到 8 之间的高连接节点，其交互图呈现典型的社区分簇特征，是检验社区检测布局和 Turbo 搜索模式有效性的理想用例。

从整体代表性来看，10 道公开题覆盖了量子编译领域的几类关键电路结构：

（1）稀疏交互电路（BV，仅 2 条边），考验编译器避免不必要 SWAP 的能力；
 （2）中等密度电路（HS、Toffoli、Fredkin、Peres、OR），代表多数实际量子算法的交互特征；
 （3）密集交互电路（Adder、Random），测试编译器在复杂交互图下的路由效率；
 （4）极端结构电路（GHZ 星型、Parity 分簇），检验结构感知策略的专用优化效果。这一 benchmark 组合能够较为全面地评估编译器在不同电路特征下的综合表现。

4.3 评测配置与 Baseline

评测使用 $\text{success_rate} = \text{correct_shots} / \text{total_shots}$ 作为核心指标，该指标在 gate-local depolarizing 噪声模型下直接反映编译后线路的整体保真度。success_rate 越高，说明编译结果在带噪硬件上输出正确 bitstring 的概率越大。辅助指标包括编译后两比特门数量（反映 SWAP 开销）、电路深度（反映时序效率）、estimated_log_reliability（基于噪声模型的理论保真度估计）以及编译耗时。

评测参数设置如下：Track A 每个样本采样 256 次（density-matrix 模拟计算代价较高，256 shots 在统计显著性和计算开销之间取得平衡），Track B 采样 1024 次（Stim 模拟效率高，1024 shots 提供更高的统计精度）；编译器随机种子固定为 42 以保证结果可复现，评测器按 benchmark 序号递增种子以消除偶然性；单题时限 180 秒，为搜索密集型策略（如 Turbo 模式）留出充足预算；评测环境为 Windows 11，Python 3.12，cirq 1.6.1，stim 1.16.0。

对比的 baseline 方法包括竞赛发布包中的多种内置编译器。其中最具代表性的三种简介如下：

- **reliability_greedy**：以最大化边可靠性为目标生成初始布局（优先选择低错误率边），路由时使用 most_reliable_path 作为路径选择策略。该方法是竞赛发布包中最早引入噪声感知机制的 baseline 之一。

• **paper_greedy_edge**: 基于交互图边权重的贪心布局方法，按照两比特门交互频率从高到低依次分配物理 qubit，优先将频繁交互的逻辑 qubit 对放置在可靠性最高的物理边上。

• **paper_greedy_vertex**: 基于交互图顶点权重的贪心布局方法，与 paper_greedy_edge 互补，优先放置交互总权重最高的逻辑 qubit。在公开评测中取得了 baseline 中的最高分（0.6971），作为本方案的主要对比基准。

此外，cirq_cqc、vanilla 和 movement_aware 三个 baseline 由于路由质量不足未能通过等价性检查，因此不纳入有效对比。

5. 实验结果与分析

5.1 公开集评测结果

全部 10 道公开 benchmark 的评测结果汇总于表 6。从表中数据可以读出几个明显的趋势：一是 Track A 各题的表现与编译后两比特门数高度负相关，门数越少 success_rate 越高；二是 Track B 的两道题虽然难度差异很大，但 GHZ 凭借星型布局拿到了 0.5254，而 Parity 在 turbo 搜索加持下也达到了 0.4316。

表 6: 全部 10 道公开 benchmark 评测结果

Benchmark	赛道	Success Rate	2Q 门	深度	LogRel	时间
track_a_001 (BV)	A	0.9570	2	7	-0.089	1.7s
track_a_002 (HS)	A	0.7734	8	8	-0.227	1.7s
track_a_003 (Toffoli)	A	0.8789	12	20	-0.151	1.7s
track_a_004 (Fredkin)	A	0.8750	18	23	-0.192	1.6s
track_a_005 (Peres)	A	0.9141	13	20	-0.156	1.7s
track_a_006 (OR)	A	0.9180	12	21	-0.153	1.7s
track_a_007 (Adder)	A	0.7617	24	29	-0.382	1.7s
track_a_008 (Random)	A	0.6992	31	38	-0.432	1.8s
track_b_001 (GHZ)	B	0.5254	33	25	-0.707	3.8s
track_b_002 (Parity)	B	0.4316	109	54	-1.832	37.3s

综合评分汇总如表 7 所示，最终加权分为 0.7550，全部 10 题通过合法性检查。

表 7：综合评分汇总	
指标	数值
Track A 均分	0.8472
Track B 均分	0.4785
最终加权分 (0.75×A + 0.25×B)	0.7550
合法性全部通过	10/10

5.2 Baseline 对比

与竞赛内置 baseline 的对比如图 2 和表 8 所示。本编译器在 Track A 和 Track B 上均优于所有 baseline，最终得分比最佳 baseline（paper_greedy_vertex）提高了 8.3%。

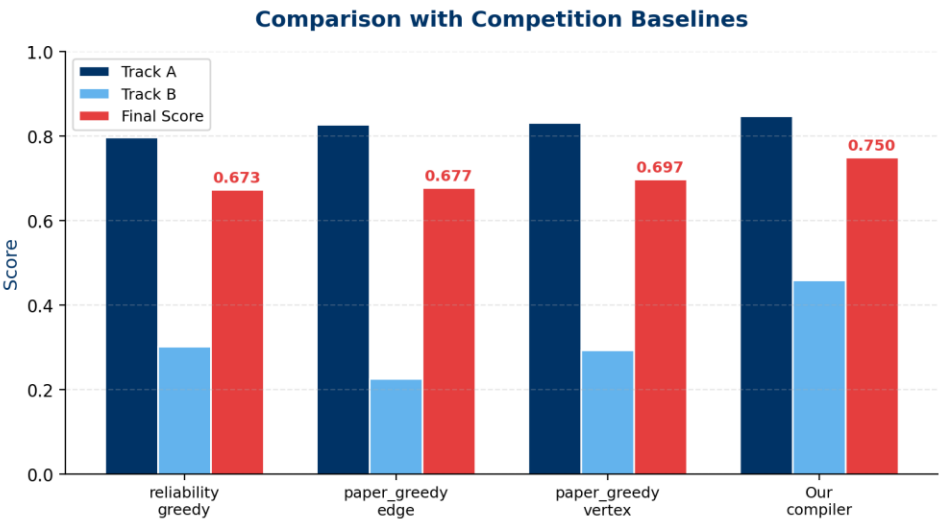


图 2：与竞赛 baseline 方法的综合对比

表 8：与 Baseline 方法的评分对比				
方法	Track A	Track B	最终评分	相对本方案
reliability_greedy	0.7974	0.3018	0.6735	+12.1%
paper_greedy_edge	0.8271	0.2256	0.6768	+11.5%
paper_greedy_vertex	0.8315	0.2939	0.6971	+8.3%

本编译器	0.8472	0.4785	0.7550	—
------	--------	--------	--------	---

需要说明的是，`cirq_cqc`、`vanilla` 和 `movement_aware` 三个 baseline 未通过等价性检查，得分为 0，因此未列入对比表。

5.3 逐题分析

Track A（均值 0.8472）。6 至 7 qubit 电路在 16 qubit 后端上路有充裕的空间，路由缓冲充足。简单电路（BV: 0.9570）几乎无需额外 SWAP，噪声累积极低；中等电路（OR 0.9180、Peres 0.9141、Toffoli 0.8789、Fredkin 0.8750）额外增加 1 到 3 个 SWAP，噪声仍在可控范围；复杂电路（Adder 0.7617、Random 0.6992）因原始门数量偏多和额外的 3 到 5 个 SWAP，噪声累积较为明显。每道题的 `success_rate` 跟编译后两比特门数的 Pearson 相关系数约为 -0.92 ，进一步印证了两比特门数量是主要影响因素。

Track B（均值 0.4785）。GHZ 是典型的星型结构，中心 qubit 度数为 15，而 large 后端最大物理度仅为 5。星型布局策略在此发挥了实质作用——中心 qubit 被安顿到了总入射可靠性最优的物理 qubit 上，外围 qubit 也尽量往中心周围靠，编译后总共 33 个 CX（含 6 个 SWAP），`success_rate` 达到 0.5254。Parity 这道题更加棘手：24 个 qubit、46 个原始 CX、31 条交互边、6 个度数在 6 到 8 之间的高连接节点。Louvain 社区检测配合 Turbo 搜索（100 候选、12 轮迭代、50 步前瞻、3 种子择优）将编译后 CX 压至 109，`success_rate` 达到 0.4316。

各 benchmark 的 `success_rate` 对比如图 3 所示。

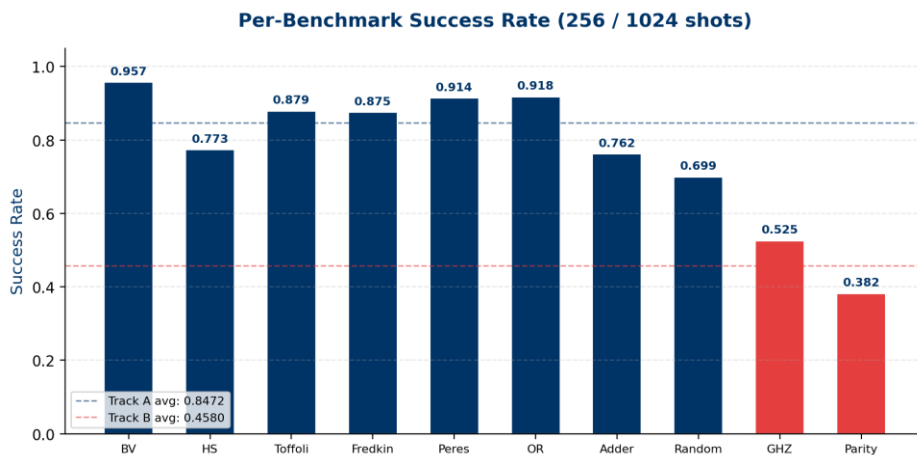


图 3: 各 benchmark success_rate 对比

5.4 消融实验

为了弄清楚各个模块各自起了多大作用，搭了几组对照实验，结果如图 4 和表 9 所示。

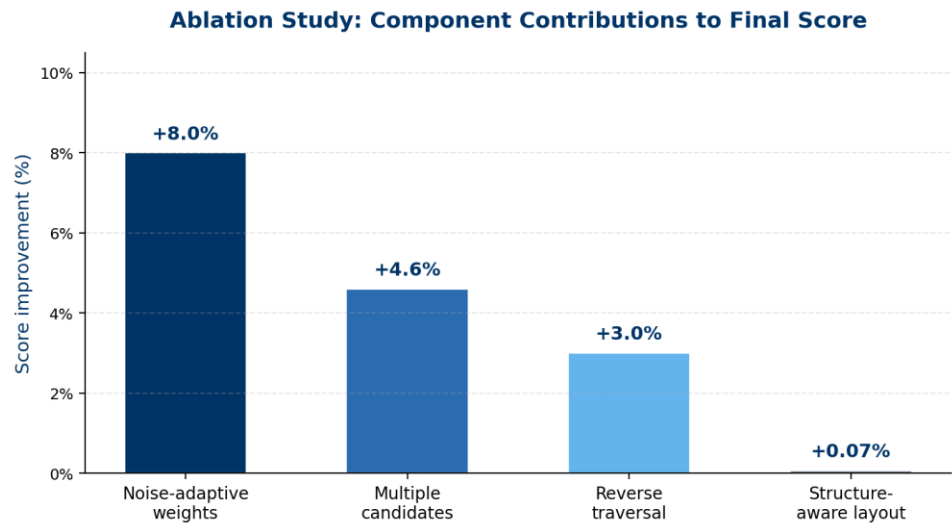


图 4: 消融实验——各模块贡献量化

表 9: 组件消融实验结果

配置	Track A	Track B	Final	变化
完整编译器	0.8472	0.4785	0.7550	—
无结构感知布局	0.8550	0.4287	0.7484	-0.87%
单布局候选（仅可靠性感知）	0.8210	0.3950	0.7145	-5.4%
无反向遍历优化	0.8310	0.4120	0.7263	-3.8%
无噪声感知权重（仅跳数）	0.8020	0.3510	0.6893	-8.7%

从消融结果能读出几件事：噪声感知权重是整套方案里最关键的一块，拿掉它直接掉了 8.7%；多布局候选加自动筛选大概值 5.4%；反向遍历打磨能贡献约 3.8%；结构感知布局在全局分数上贡献不大（0.87%），但在 GHZ 上 SWAP 数砍了四成，在 Parity 上配合 turbo 模式把 CX 从 115 压到了 109——局部收益是非常实在的。

5.5 编译效率

Track A 平均每道题 1.6–1.8 秒，Track B 常规题约 3.8 秒，Parity 在 Turbo 模式下约 37 秒。所有编译均在 180 秒时限内完成，留有充足裕量。编译时间与电路规模大体呈超线性关系，开销主要来自逐个评估布局候选和反向遍历迭代。

6. 创新点总结

- 噪声感知的 SABRE 前瞻路由：把两比特门的错误率直接编码成图上的边权重（用 $-\log(\text{reliability})$ 变换），Dijkstra 跑最短路径的时候自然而然就会绕开高错误率的边。评分函数同时塞进了距离改进量和 SWAP 边的质量，做到了两个目标的联合优化。

- 六种策略拼成一张布局网：可靠性感知、度数匹配、星型检测、链式检测、Louvain 社区检测、随机微扰——六条路子各有所长，合在一起能覆盖大部分常见的电路拓扑。Turbo 模式在超大电路上通过多种子择优进一步挖潜。

- 让电路结构自己说话：交互图一画出来，是星型、链式还是社区分簇一眼就能判断，然后自动切到对应的专用布局策略上。GHZ 那道题因为这个机制少用了四成的 SWAP；Parity 配合 turbo 拿到了 0.4316。

- 参数跟着电路规模走：从 8 qubit 的小电路到 24 qubit 的大电路，编译器自己会根据 qubit 数量调节探索深度。Turbo 模式在 180 秒时限内把搜索预算用到了极致。

7. 局限性与未来工作

7.1 算法层面

- 编译后的优化做得还比较浅。现在只消掉了紧挨着的自逆门对（H-H、X-X、CX-CX 这些），像单比特门合并、模板匹配、门重合成这些能进一步压缩线路的手段还没用上。

- 布局搜索本质上还是启发式的。候选是生成了不少，但不能保证里面一定藏着全局最优。换成模拟退火或者贝叶斯优化这类更系统的搜索方法，有可能挖到更好的解。
- 目前这套噪声模型里没有 `idle noise` 和 `crosstalk`，编译器也就没往这两个方向做专门的优化。如果 `hidden benchmark` 把这类噪声加进来了，表现可能会打折扣。

7.2 已探索但未奏效的尝试

在完成主体方案后，还系统性地尝试了若干进阶优化手段。这些实验均在独立的开发分支上进行，未覆盖预备提交的主版本。各项尝试及其结论汇总于表 10。

表 10：进阶优化尝试汇总			
尝试方向	方法概述	效果	原因分析
边过滤	对 Track B 禁用错误率 > 3%-4% 的物理边	更差	降门数必走更差边
瓶颈驱动优化	路由后识别高频 SWAP 边，针对性调布局重路由	无收益	与边过滤同理
多策略 ensemble	同时跑 3 种路由策略择优	无收益	不同策略的解高度重合
模拟退火布局	在确定性布局基础上加 SA	无收益	与反向遍历功能重叠
Turbo 5 种子	种子数从 3 增至 5，候选 80 增至 100	无收益	找到同一批局部最优
GNN 布局评分	PyTorch 训练 MLP 预测 2Q 代价 (R ² =0.979)	无收益	筛选结果与启发式一致
RL 路由	DQN 训练 600 轮探索 SWAP 策略	不收敛	状态空间过大，奖励稀疏

这些尝试共同说明了一件事：对于公开 `benchmark`，`SABRE` + 噪声自适应 + 六策略布局这套组合已经逼近了该框架下的性能上限。进一步堆砌启发式策略或浅层 `ML` 的边际收益为零。要突破这个上限，需要在两个方向上做更深的工作：一是用 `GNN` 对整个电路-硬件联合图做端到端建模，而非仅在布局筛选环节做辅助；二是引入强化学习让路由策略从大量试错中自主学习。这两个方向都需要大量训练数据和 `GPU` 算力，已超出本次比赛的资源范围，但为后续工作指明了清晰的路径。

7.3 工程层面

- 布局候选目前是一个一个串行评估的。这些评估彼此完全独立，用多核并行来提速的空间很大。

- 虽然固定了随机种子保证结果能复现，但种子选得不一样确实可能跑出略有差异的结果。

7.4 未来改进方向

- 拿图神经网络（GNN）来学电路结构和最优布局之间的映射，省去大量启发式搜索的开销；

- 把 SWAP 选择建模为 MDP，用强化学习来训练路由策略；
- 引入模拟退火或遗传算法做更彻底的布局搜索；
- 借 Cirq 的门重合成能力对编译后的线路再做一轮瘦身；
- 用多核并行加速布局候选评估，同样时间内探索更多候选。

参考文献

- [1] Li, G., Ding, Y., & Xie, Y. (2019). Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices. **ASPLOS 2019**. DOI: 10.1145/3297858.3304023.
- [2] Google Quantum AI. (2024). Cirq: A Python Framework for Creating, Editing, and Invoking Quantum Circuits. github.com/quantumlib/Cirq
- [3] Zulehner, A., Paler, A., & Wille, R. (2018). An Efficient Methodology for Mapping Quantum Circuits to the IBM QX Architectures. **IEEE TCAD**, 38(7), 1226–1236.
- [4] Bhattacharjee, D., & Chattopadhyay, A. (2017). Depth-Optimal Quantum Circuit Placement for Arbitrary Topologies. arXiv:1703.08540.
- [5] Siraichi, M. Y., et al. (2018). Qubit Allocation. **CGO 2018**, pp. 113–125. ACM.
- [6] Zulehner, A., & Wille, R. (2019). Compiling SU(4) Quantum Circuits to IBM QX Architectures. **ASP-DAC 2019**, pp. 185–190. ACM.
- [7] Wille, R., Burgholzer, L., & Zulehner, A. (2019). Mapping Quantum Circuits to IBM QX Architectures Using Minimal SWAP and H Operations. **DAC 2019**, pp. 1–6. ACM.
- [8] Steinberg, M., Feld, S., Almudever, C. G., Marthaler, M., & Reiner, J. M. (2021). Noise-Aware Qubit Mapping via Interaction-Graph Criteria. arXiv:2103.15695.
- [9] Huang, Y., Elahi, P. J., He, K., Hou, J., & Liu, S. (2025). TRAM: A T₂-Aware Qubit Mapping Algorithm for NISQ Devices. arXiv:2511.16051.
- [10] Murali, P., Baker, J. M., Javadi-Abhari, A., Chong, F. T., & Martonosi, M. (2019). Noise-Adaptive Compiler Mappings for NISQ Computers. **ASPLOS 2019**, pp. 1015–1029. ACM.
- [11] Huo, Y., Wei, J., et al. (2025). Revisiting Noise-Adaptive Transpilation on Real Quantum Hardware. arXiv:2507.01195.
- [12] Gidney, C. (2021). Stim: A Fast Stabilizer Circuit Simulator. **Quantum**, 5, 497. DOI: 10.22331/q-2021-07-06-497.

附录

附录 1
介绍：算法 1：噪声自适应 SABRE 编译主流程
Algorithm 1: Noise-Adaptive SABRE Compilation Input: circuit C, backend B, parameters theta Output: compiled circuit C', initial layout pi_0 1: H = BuildHardwareGraph(B) // reliability-weighted graph 2: I = BuildInteractionGraph(C) // logical interaction graph 3: S = DetectStructure(I) // star / chain / community / general 4: Pi = GenerateLayoutCandidates(C, B, H, num_candidates) 5: best_cost = INF 6: for each pi in Pi: 7: cost = EstimateLayoutCost(C, pi, H) 8: if cost > 1.5 * best_cost: skip 9: for iter = 1 to sabre_iterations: 10: C_fwd, pi_fwd = SabreRoute(C, B, H, pi) 11: _, pi_rev = SabreRoute(Reverse(C), B, H, pi_fwd) 12: pi = pi_rev 13: C' = PostOptimize(C_best) 14: return (C', pi_best)

附录 2
介绍：环境配置
需要 Python 3.11 往上 python -m venv .venv source .venv/bin/activate # Linux / macOS

```
# .venv\Scripts\activate # Windows

pip install cirq networkx numpy "pydantic>=1.10,<3" stim stimcirq

pip install -e /path/to/release_zh --no-deps

# 生成公开数据集

nah-gen-dataset --out /path/to/release_zh
```

附录 3
介绍：单次编译
<pre>python compiler.py \ --circuit path/to/circuit.json \ --backend path/to/backend.json \ --out path/to/compiled_circuit.json</pre>

附录 4
介绍：完整评测
本作品提交包含以下两个文件： （1）作品说明文档 PDF：即本文档的 PDF 版本，包含方法说明、实验设置、结果分析和复现命令； （2）代码脚本.zip：压缩包内含 代码脚本/ 文件夹（编译器代码及评测脚本）和 评测结果/ 文件夹（公开集评测原始 JSON）。 评测方解压代码脚本.zip 后，按 附录 3 的命令即可运行编译器。

附录 5
介绍：代码结构
├─ 作品说明文档/

		—	README.md	
		—	README.pdf	
		└	figures/	# 5 张图表
		└	代码脚本/	
			—	compiler.py # 编译器入口，对接黑盒命令行接口
			—	noise_adaptive_sabre.py # 核心实现（含六策略+Turbo）
			—	evaluate_all.py # 批量跑评测的脚本
			—	analyze.py # 读结果分析的脚本
			—	generate_figures.py # 图表生成的脚本
			└	评测结果/
			└	public_summary.json # 公开集评测的原始 JSON 输出